



Bohdan DYSHLEVYY
Etudiant de BUT Réseaux et
Télécommunications
IUT NICE CÔTE-D'AZUR
PROMOTION BUT3
Année 2025-2026

Maître d'apprentissage :
Damien SAUCEZ
Chercheur INRIA
Centre Inria d'Université Côte
d'Azur

Configuration automatique du réseau expérimental SLICES-RI

au Centre Inria d'Université Côte d'Azur, Équipe-projet DIANA

Tuteur pédagogique :
Laurent KWIATKOWSKI
Maître de Conférences
à l'Université Côte d'Azur
IUT NICE CÔTE D'AZUR



Trois missions pour rendre SLICES-RI plus exploitable

1

Inventorier

Découvrir et publier automatiquement les ressources matérielles

2

Expérimenter

Déployer des environnements JupyterHub reproductibles et isolés

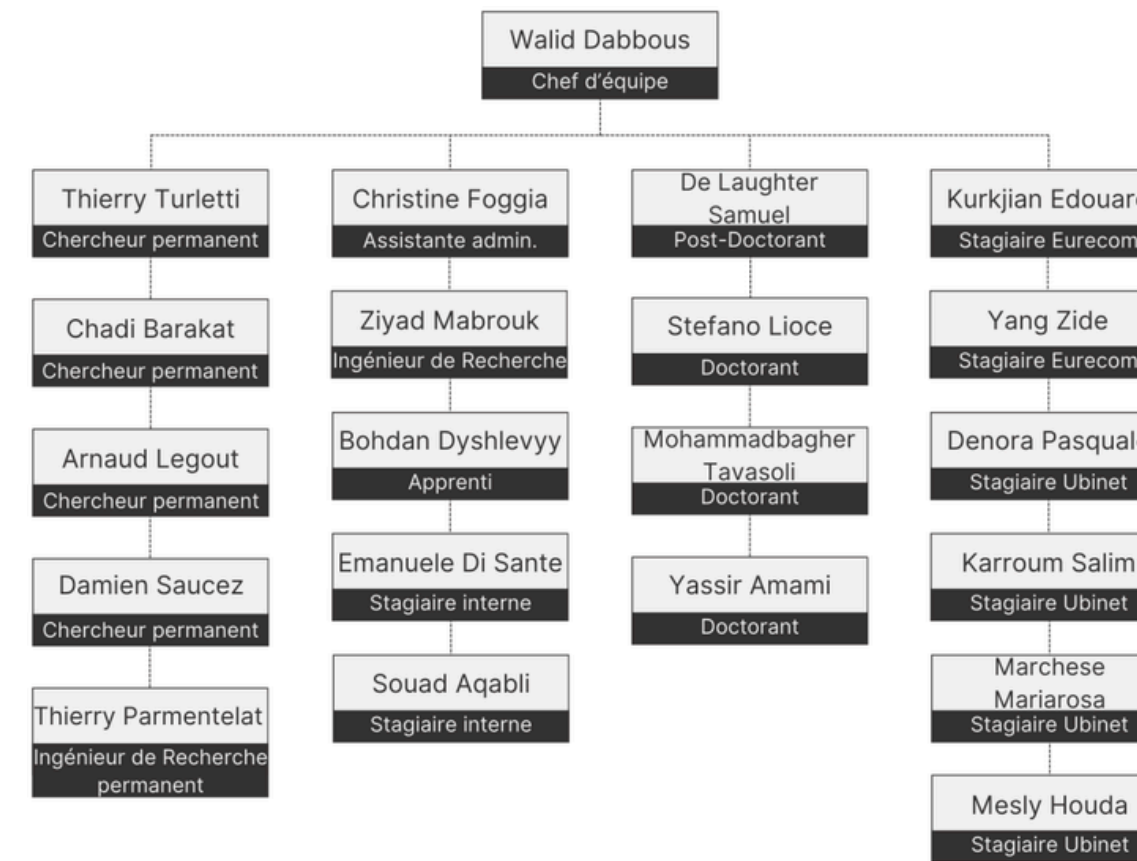
3

Mesurer

Évaluer le partage d'un GPU H100 avec NVIDIA MIG

Un fil conducteur : automatisation, isolation et reproductibilité

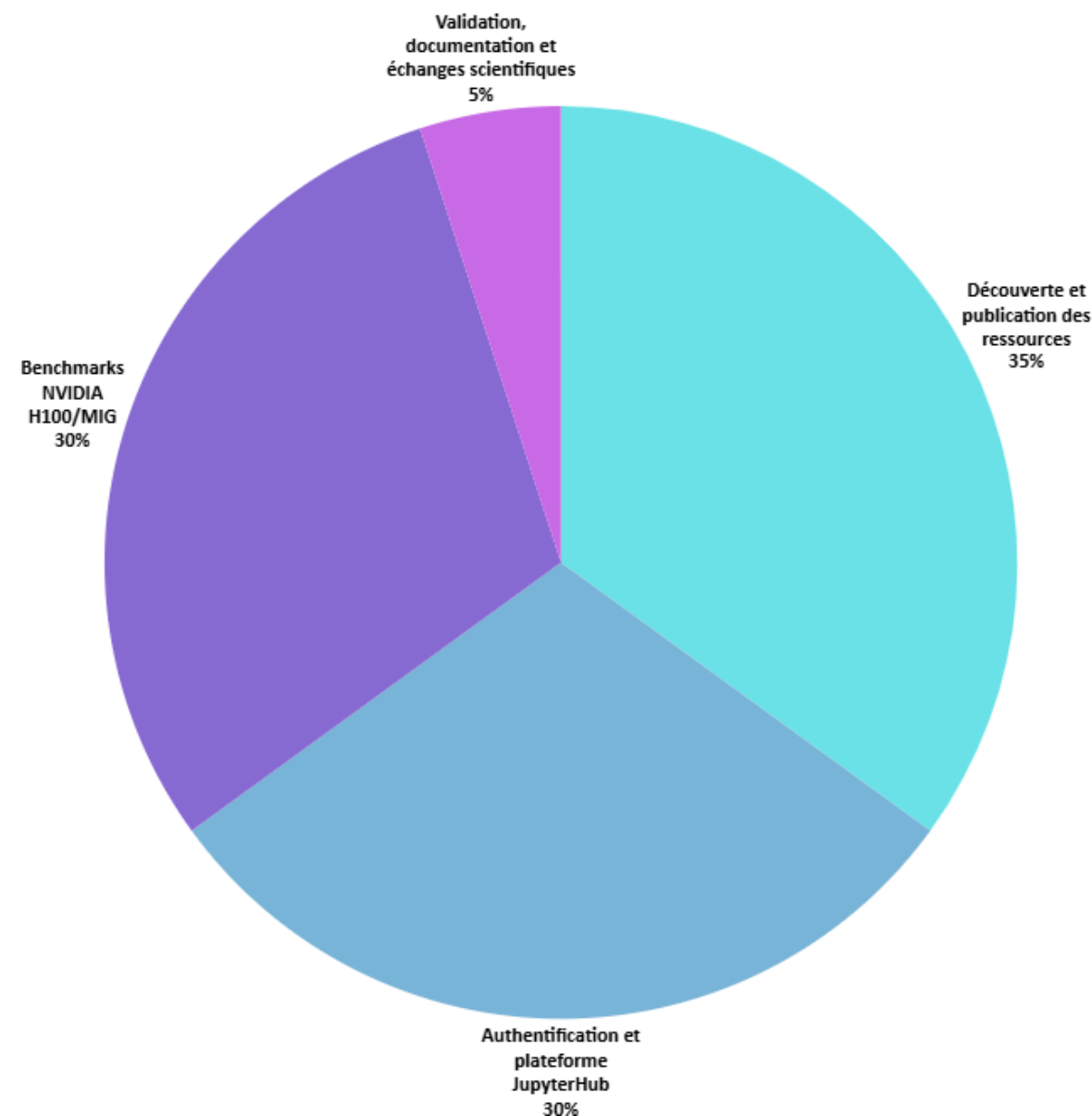
Inria et DIANA : le cadre de l'alternance



- Centre Inria d'Université Côte d'Azur, Sophia Antipolis
- Équipe-projet DIANA : réseaux modernes, systèmes distribués et expérimentation
- Missions menées dans le cadre de projets européens, dont SLICES-RI et DigitAfrica

Une même problématique relie les trois missions

Comment améliorer l'automatisation, la fiabilité et l'exploitation d'une infrastructure distribuée, tout en garantissant des expérimentations reproductibles ?



- **35 %** : découverte et publication des ressources
- **30 %** : authentification et plateforme JupyterHub
- **30 %** : benchmarks NVIDIA H100/MIG
- **5 %** : validation, documentation et échanges scientifiques

Un catalogue fiable exige un inventaire automatisé

- Les ressources SLICES-RI évoluent régulièrement
- Une description incomplète empêche de choisir le bon nœud
- Une mise à jour manuelle est lente et source d'incohérences

```
Infos de slices-su-vm sauvegardées
Nœud slices-su-vm libéré
=== Traitement du nœud sopnode-bm101 ===
Attente 2 minutes pour sopnode-bm101...
Warning: Permanently added 'sopnode-bm101' (ED25519) to the list of known hosts
Warning: Permanently added 'sopnode-bm101' (ED25519) to the list of known hosts
Infos de sopnode-bm101 sauvegardées
Nœud sopnode-bm101 libéré
=== Traitement du nœud sopnode-bm102 ===
Attente 2 minutes pour sopnode-bm102...
Warning: Permanently added 'sopnode-bm102' (ED25519) to the list of known hosts
Warning: Permanently added 'sopnode-bm102' (ED25519) to the list of known hosts
Infos de sopnode-bm102 sauvegardées
Nœud sopnode-bm102 libéré
=== Traitement du nœud sopnode-f1 ===
Attente 2 minutes pour sopnode-f1...
Warning: Permanently added 'sopnode-f1' (ED25519) to the list of known hosts.
Warning: Permanently added 'sopnode-f1' (ED25519) to the list of known hosts.
Infos de sopnode-f1 sauvegardées
Nœud sopnode-f1 libéré
=== Traitement du nœud sopnode-f2 ===
Attente 2 minutes pour sopnode-f2...
```

Objectif : réserver un nœud, collecter son matériel, normaliser les données puis publier les métadonnées sans intervention humaine.

La chaîne prend en charge tout le cycle d'un nœud

Liste des nœuds à inventorier



Réservation (POS)



Démarrage / Bootstrap



Vérification de l'accès SSH



Collecte des informations



Génération des fichiers

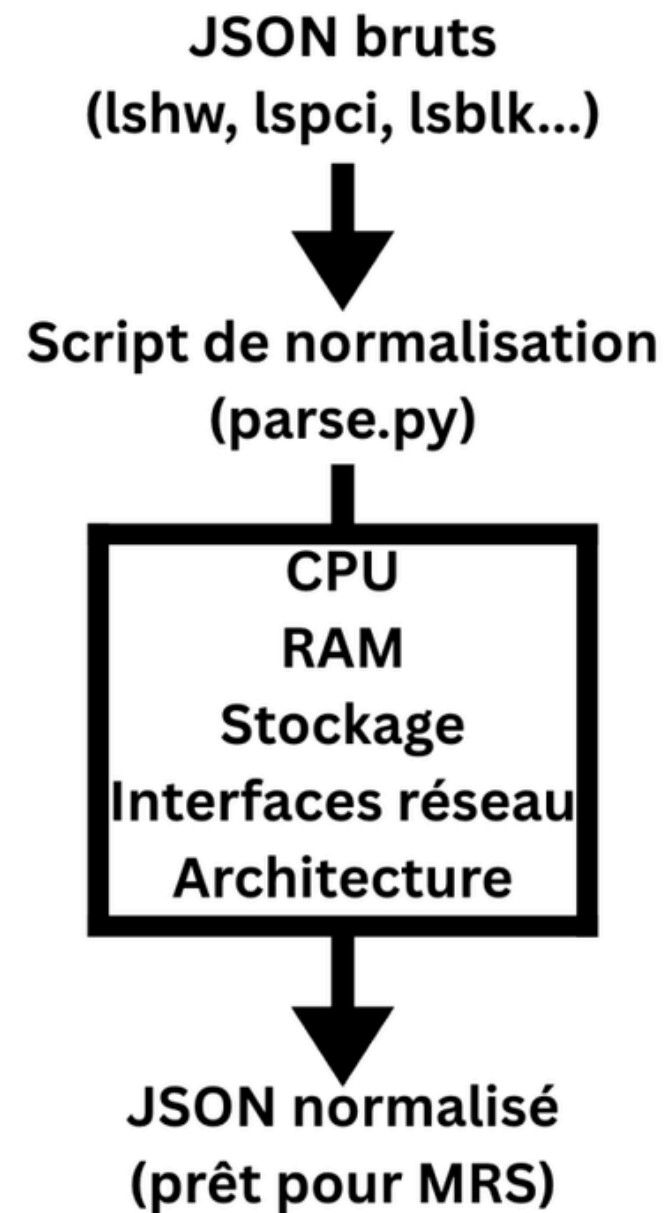
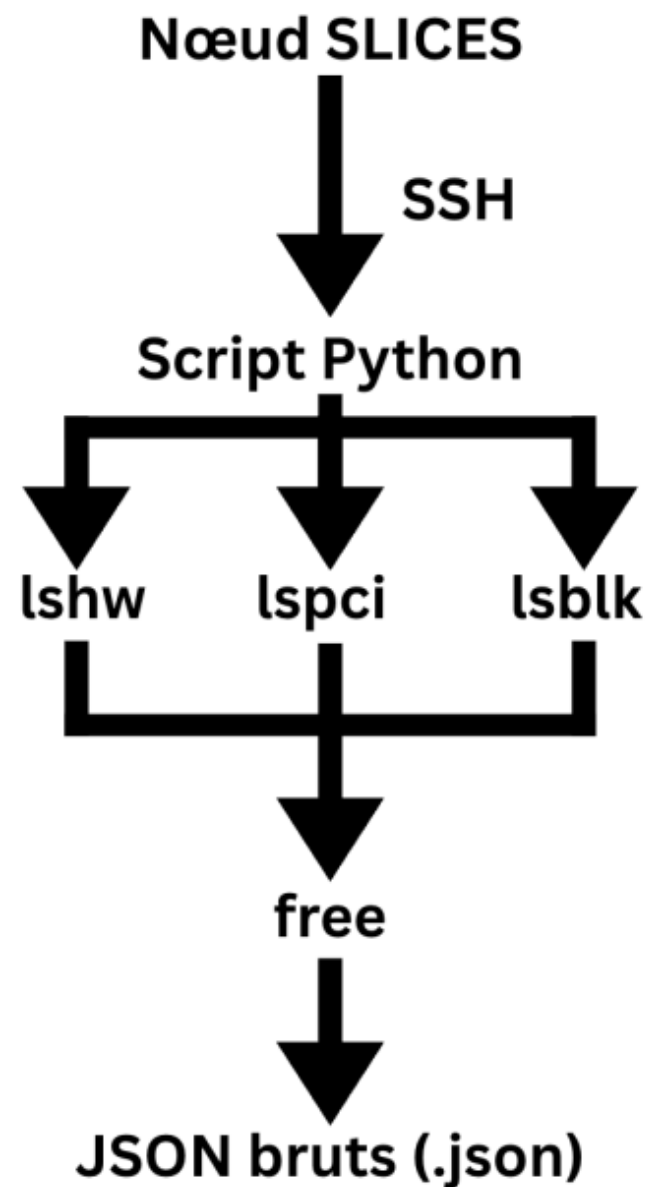


Libération du nœud (POS)

- Allocation avec le service POS
- Démarrage ou réinitialisation automatique
- Tentatives SSH jusqu'à disponibilité complète
- Collecte, génération des fichiers puis libération

Le traitement séquentiel respecte les limites de réservation et remet rapidement chaque nœud à disposition.

Plusieurs outils fiabilisent la description du matériel



lshw

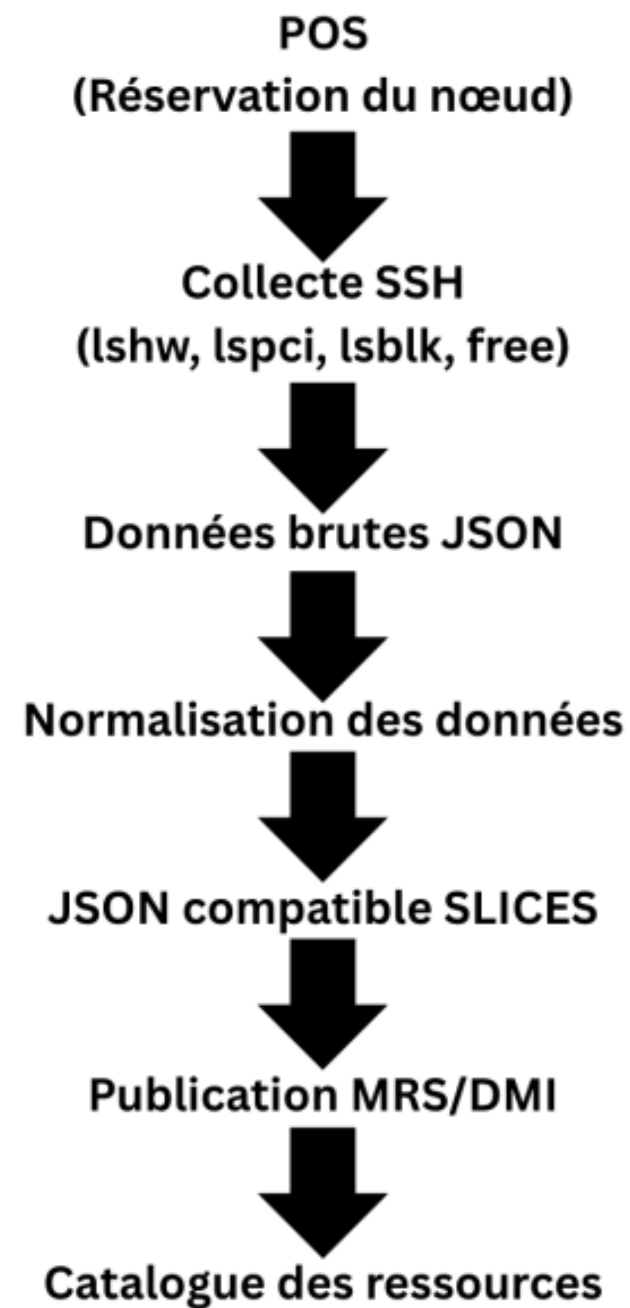
lspci

lsblk

free

Chaque outil
complète ou vérifie
les autres.

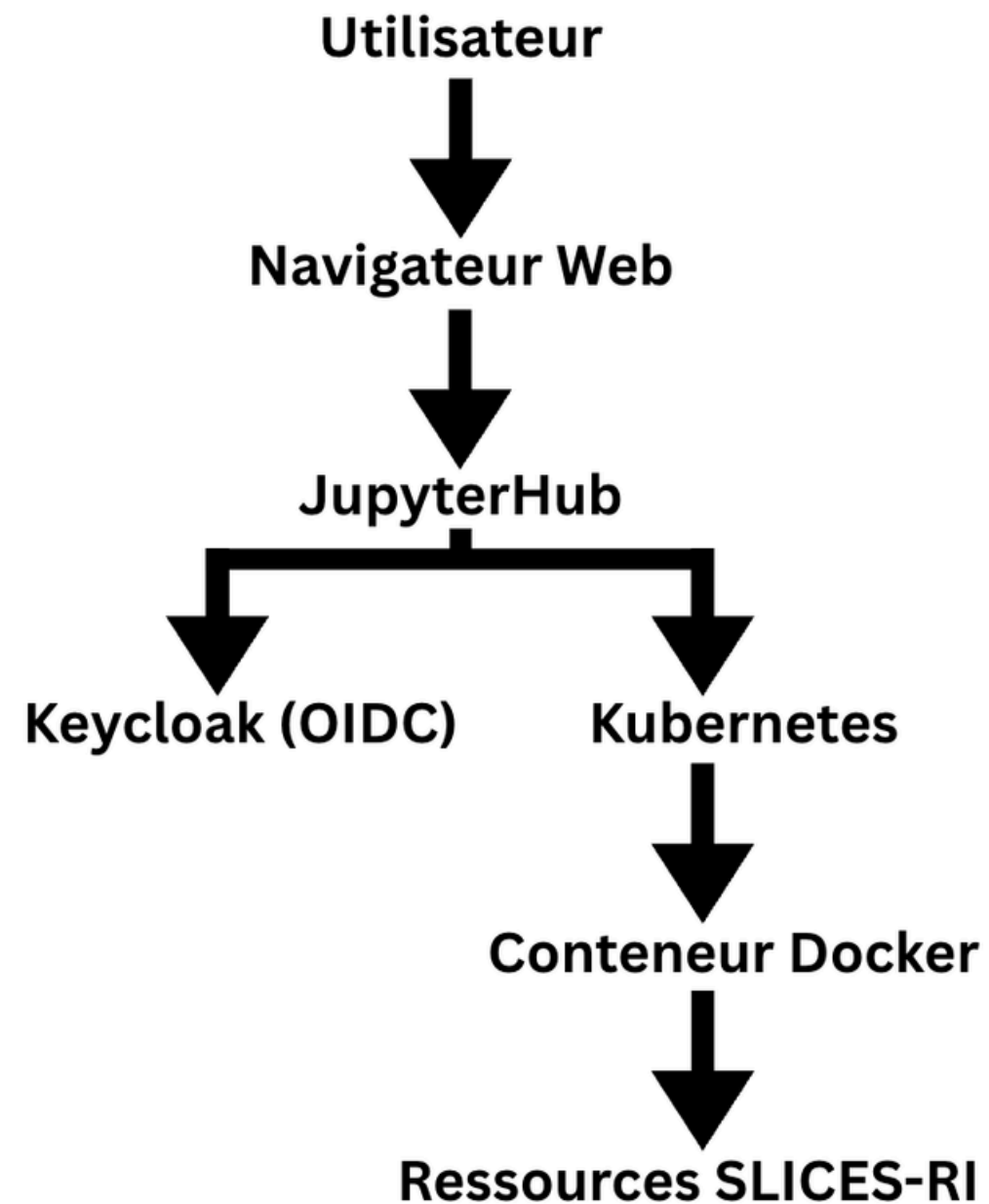
Résultat : une chaîne rejouable jusqu'au catalogue



- Métadonnées homogènes pour tous les nœuds
- Publication automatique dans MRS et DMI
- Journalisation des erreurs sans bloquer les autres ressources
- Relance possible après toute évolution de l'infrastructure

Une machine défailante ne doit plus empêcher la mise à jour de l'ensemble du catalogue.

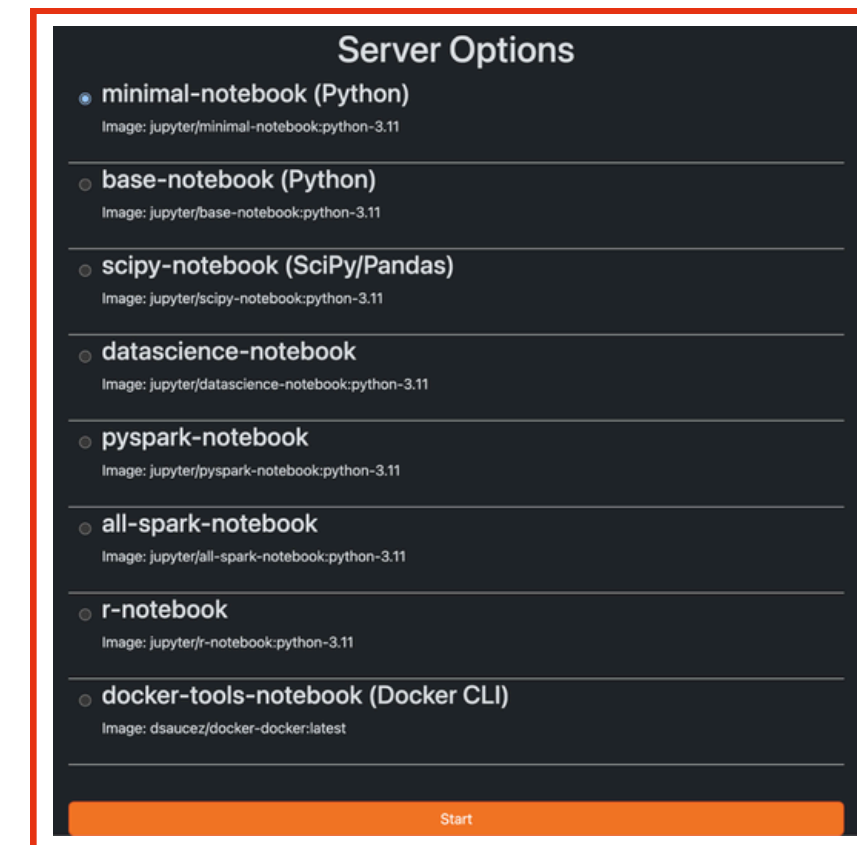
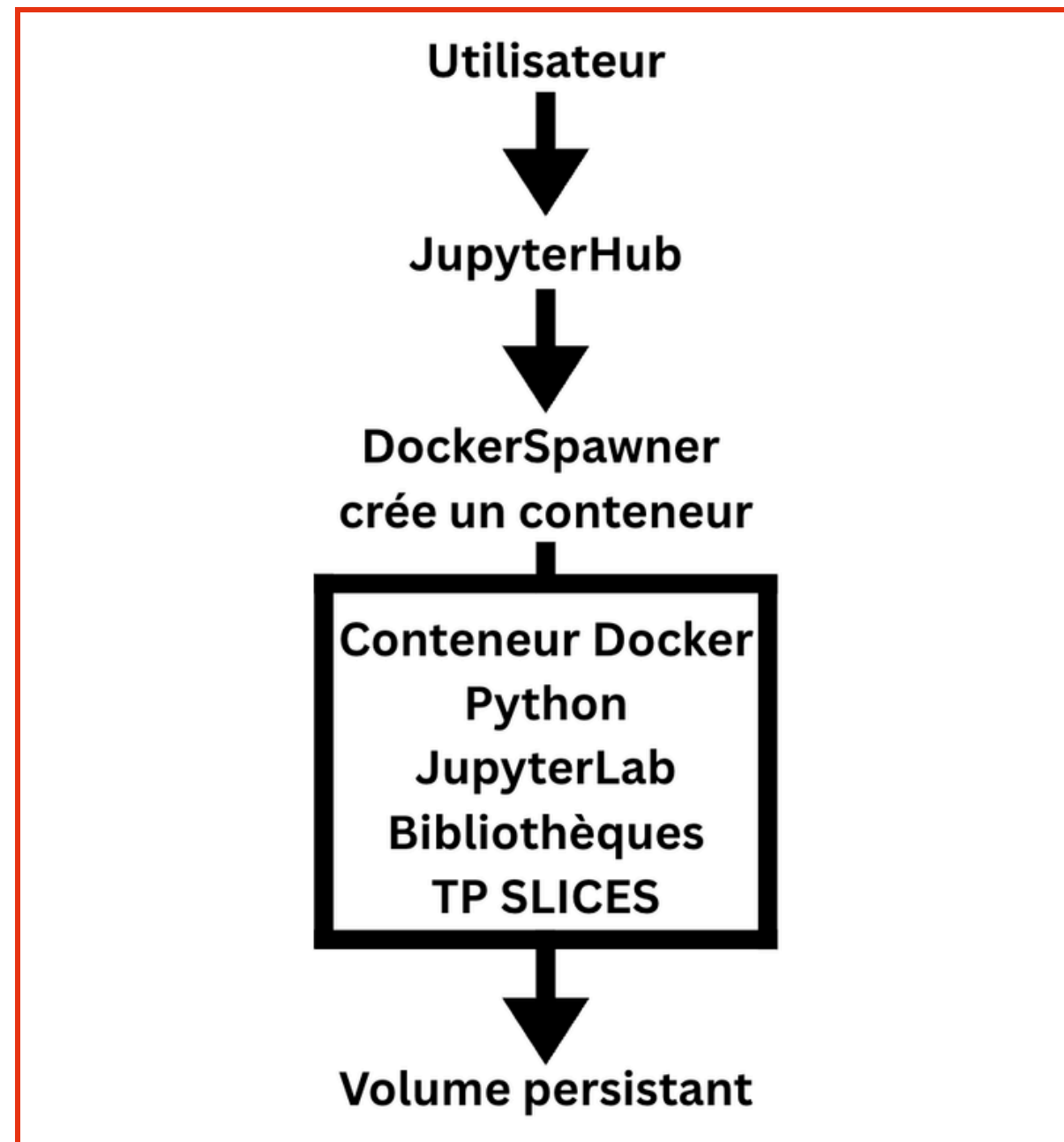
JupyterHub rend les ressources utilisables



- Environnement JupyterHub créé à la demande
- Images logicielles prêtes à l'emploi
- Conteneurs et stockage persistants
- Keycloak et OpenID Connect pour identifier les utilisateurs

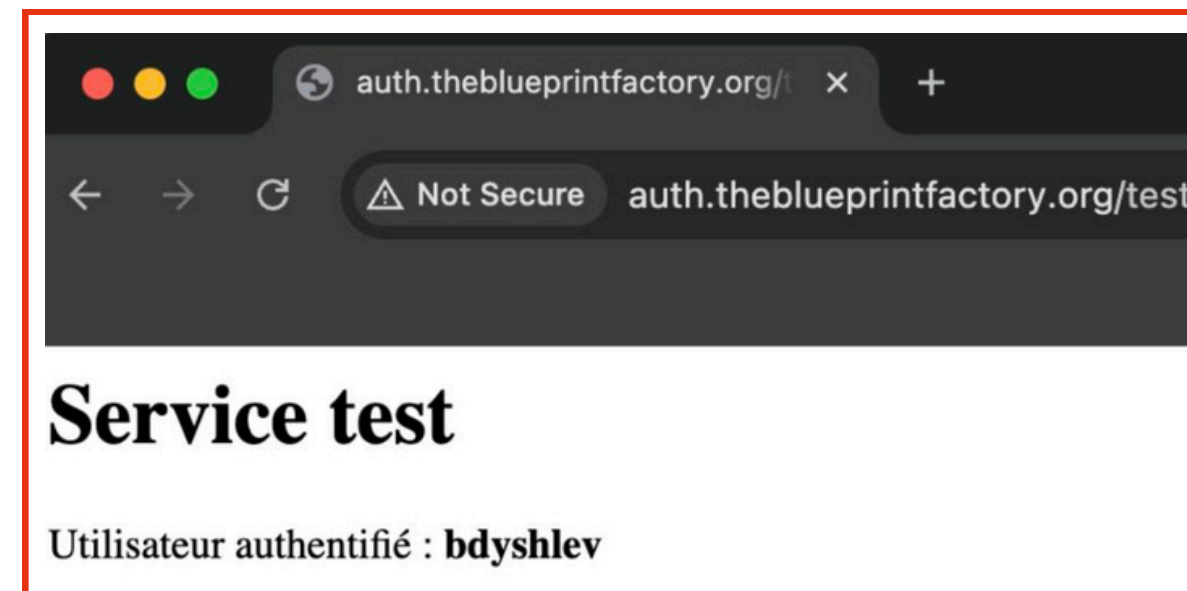
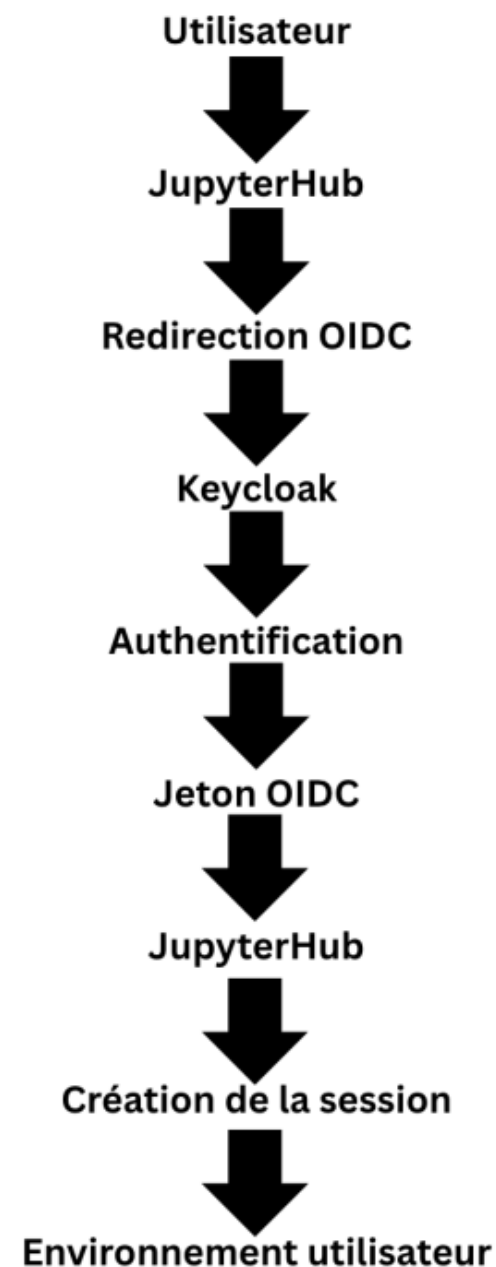
But : fournir le même environnement logiciel à chaque utilisateur, depuis un simple navigateur.

JupyterHub crée un environnement isolé à la demande



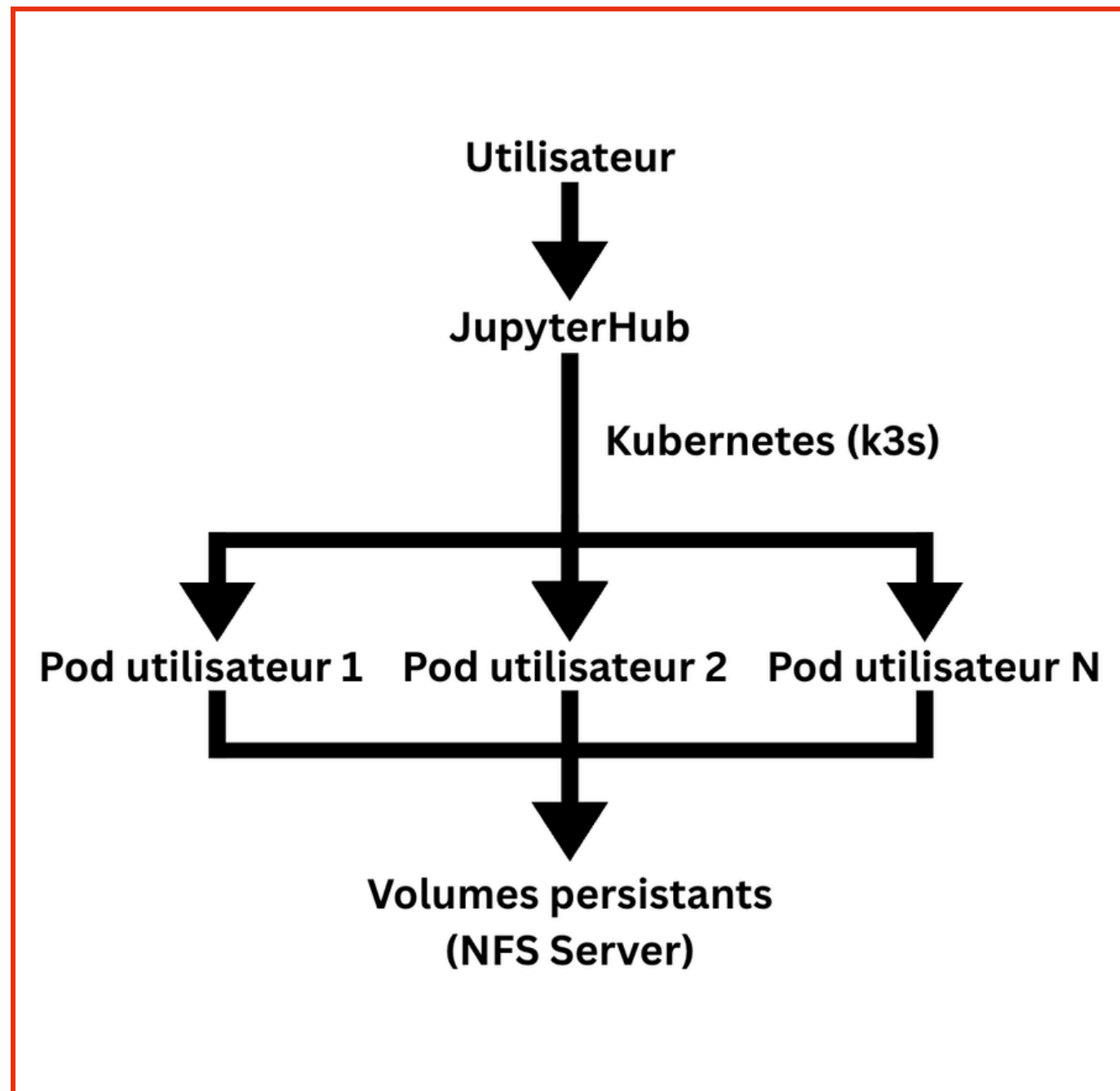
- Images adaptées : Python, data science, Spark ou besoins spécifiques
- Volumes persistants pour conserver notebooks et jeux de données
- Liste des images générée depuis YAML et Jinja2

Keycloak centralise les identités et accepte plusieurs IdP



- Plusieurs fournisseurs d'identité peuvent être raccordés
- Les applications ne stockent plus les comptes localement
- Les secrets clients sont générés automatiquement

Kubernetes permet de gérer plusieurs utilisateurs



- K3s : cluster Kubernetes léger
- KubeSpawner : un pod par utilisateur
- NFS : volumes persistants partagés
- Redémarrage automatique en cas de panne

Le cycle de vie des environnements n'est plus géré manuellement sur une seule machine.

L'automatisation ne suffit pas sans isolation

Avant : socket Docker de l'hôte

```
jovyan@c975e2f78692:~$ ls -la /var/run/docker.sock && docker ps
srw-rw-rw- 1 root root 0 Jan 26 09:21 /var/run/docker.sock
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
c975e2f78692   local/notebook-docker-tools:latest  "tini -g -- start-no-"  25 seconds ago Up 25 seconds (healthy)  8888/tcp      jupyter-bdyshlev
89fb37f27186   2a1b1fcb0728                        "tini -g -- start-no-"  17 hours ago  Up 17 hours (healthy)   8888/tcp      keycloak_test-docker-tools-n
otebook-1
b2da70fb33b0   local/notebook-oai-tools:latest     "tini -g -- start-no-"  17 hours ago  Up 17 hours (healthy)   8888/tcp      keycloak_test-oai-tools-note
book-1
a7a75cb89536   jupyterhub/jupyterhub:4.1          "bash -lc 'apt-get ..." 19 hours ago  Up About a minute      192.0.2.1:8000->8000/tcp  keycloak_test-jupyterhub-1
f9e51ff83a1f   keycloak_test-apache               "/bin/bash -lc 'echo..." 19 hours ago  Up 19 hours              192.0.2.1:80->80/tcp      keycloak_test-apache-1
ae94af72fdb0   quay.io/keycloak/keycloak:26.5.0    "/opt/keycloak/bin/k..." 19 hours ago  Up 19 hours (unhealthy)  192.0.2.1:8080->80/tcp   keycloak_test-keycloak-1
6e0396812532   postgres:16                         "docker-entrypoint.s..." 19 hours ago  Up 19 hours (healthy)   5432/tcp        keycloak_test-postgres-1
97b2e9e9d453   redis:7-alpine                     "docker-entrypoint.s..." 19 hours ago  Up 19 hours (healthy)   6379/tcp        keycloak_test-redis-1
```

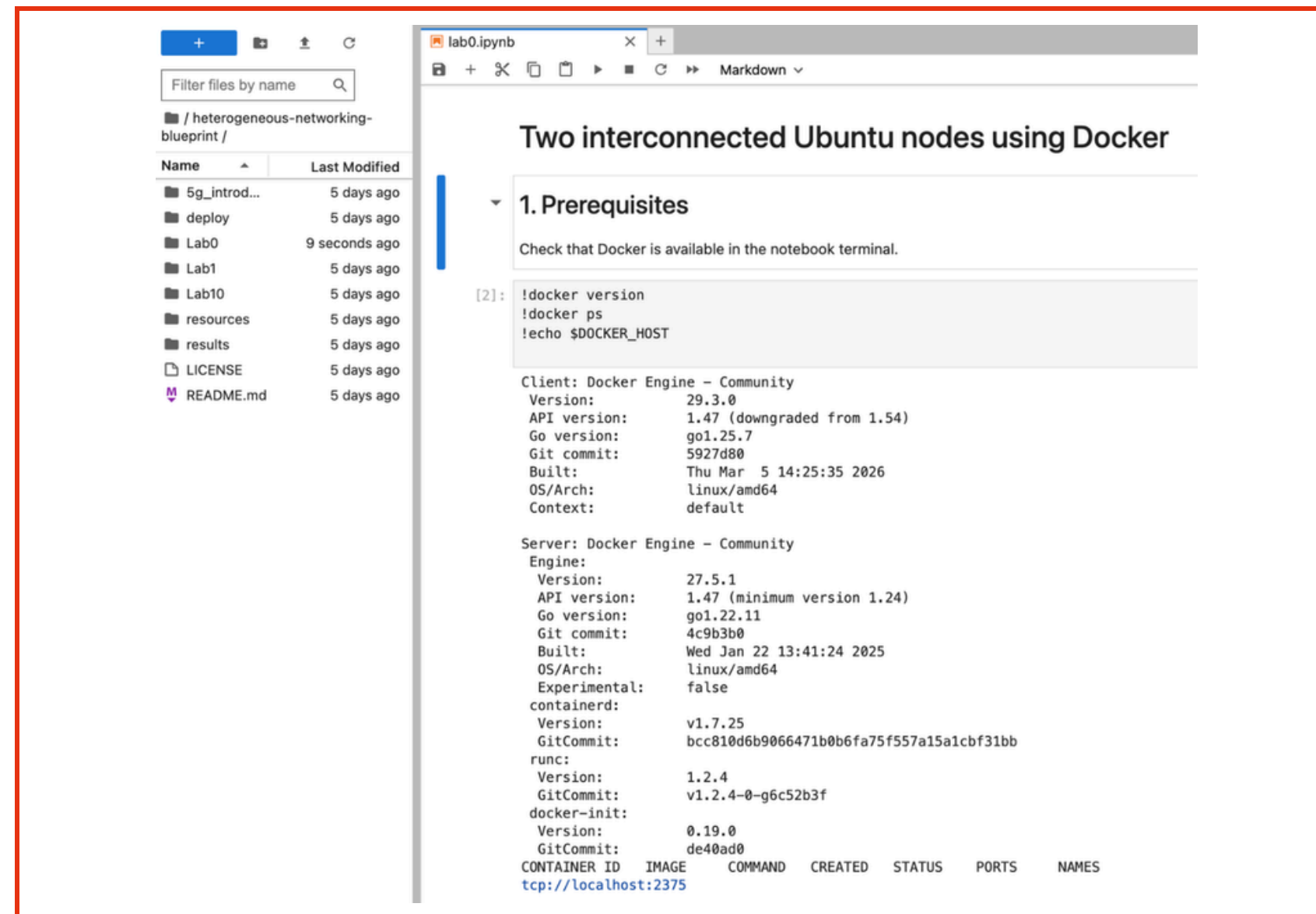
Un utilisateur pouvait voir les conteneurs des autres sessions.

Après : démon Docker privé

```
jovyan@jupyter-bdyshlev:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
jovyan@jupyter-bdyshlev:~$ █
```

Docker-in-Docker isole le moteur et les conteneurs de chaque pod.

La plateforme a été validée dans des usages réels



≈ 80

participants

Kenya (Nairobi)
Tunisie (Tunis)
Viêtnam (Hanoi)

Plateforme déployée en production
pour les expérimentations et les
travaux pratiques.

Le partage du cluster impose de vérifier l'isolation

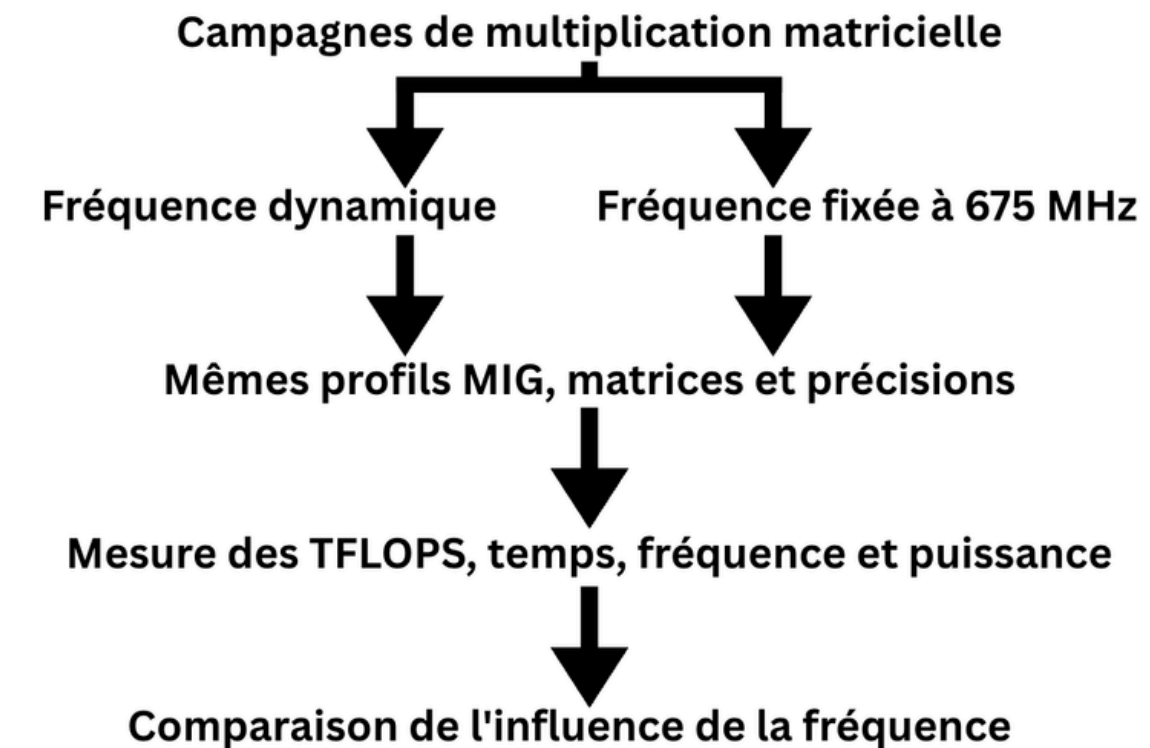
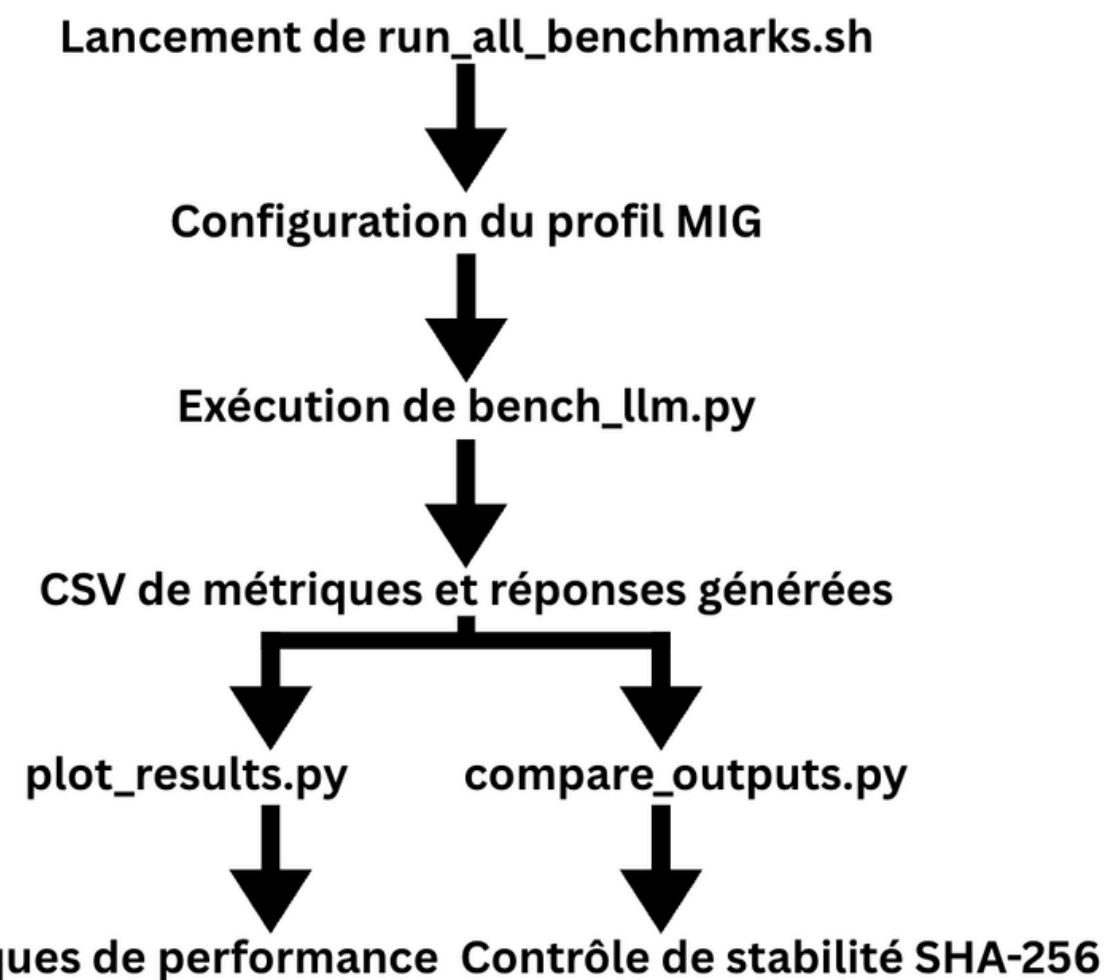
```
[(venv) root@sopnode-f4:~/llm-benchmark# nvidia-smi mig -lgip]
```

GPU instance profiles:								
GPU	Name	ID	Instances Free/Total	Memory GiB	P2P	SM CE	DEC JPEG	ENC OFA
0	MIG 1g.12gb	19	0/7	10.75	No	16 1	1 1	0 0
0	MIG 1g.12gb+me	20	0/1	10.75	No	16 1	1 1	0 1
0	MIG 1g.24gb	15	0/4	21.62	No	26 1	1 1	0 0
0	MIG 2g.24gb	14	0/3	21.62	No	32 2	2 2	0 0
0	MIG 3g.47gb	9	0/2	46.38	No	60 3	3 3	0 0
0	MIG 4g.47gb	5	0/1	46.38	No	64 4	4 4	0 0
0	MIG 7g.94gb	0	0/1	93.12	No	132 8	7 7	0 1

- CPU, mémoire, stockage et GPU sont partagés entre les utilisateurs
- Le H100 NVL de 94 Go est une ressource coûteuse et très demandée
- Six profils étudiés : de 1g.12gb à 7g.94gb
- MIG fournit des instances avec mémoire et unités de calcul dédiées

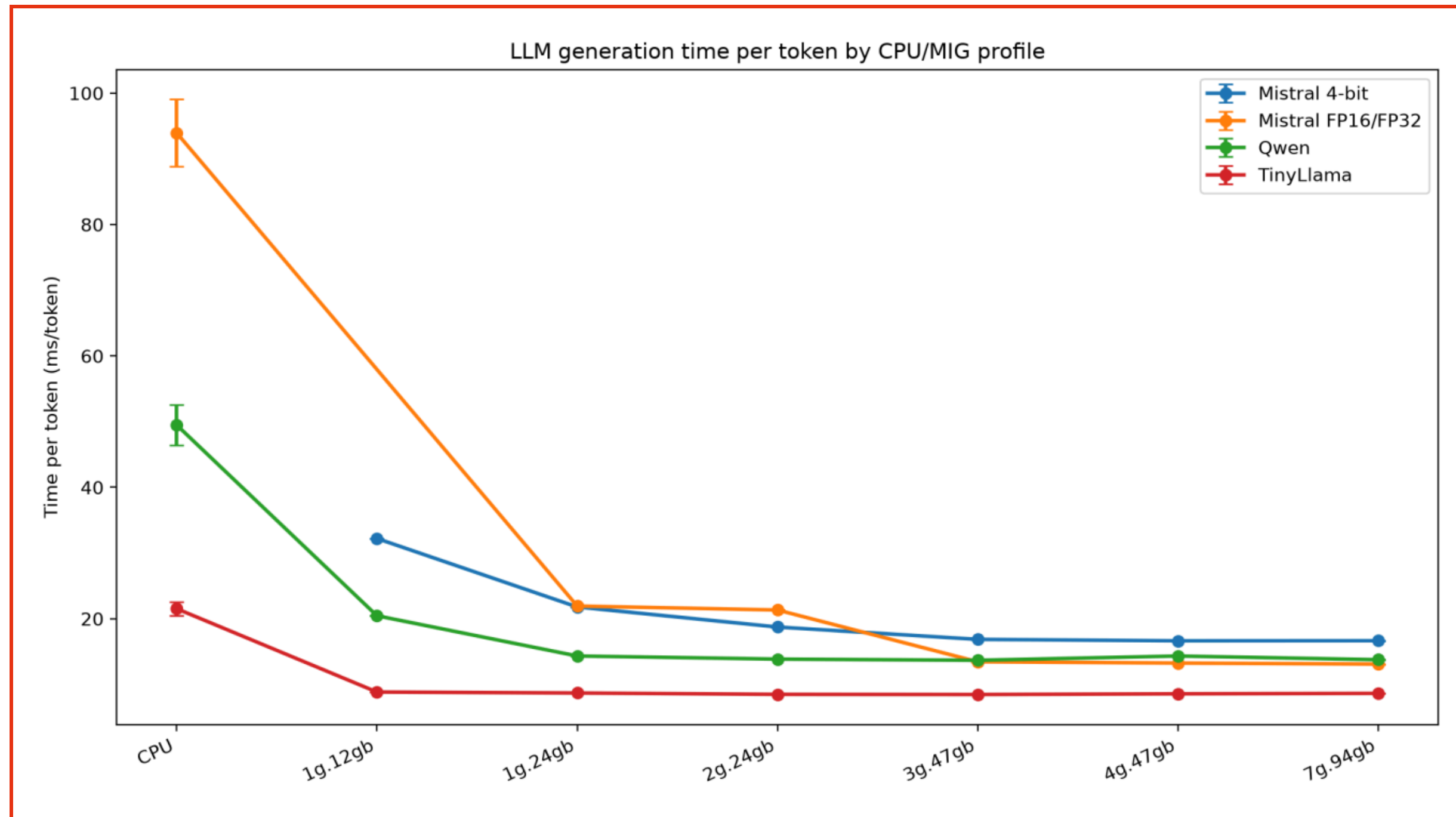
Question : les performances augmentent-elles proportionnellement à la taille du profil ?

Deux benchmarks séparent usage réel et puissance brute



- LLM : TinyLlama, Qwen et Mistral, 256 tokens, 10 répétitions
- Matrices : FP16, FP32 strict, TF32 et FP64
- Campagne dynamique puis fréquence commune de 675 MHz

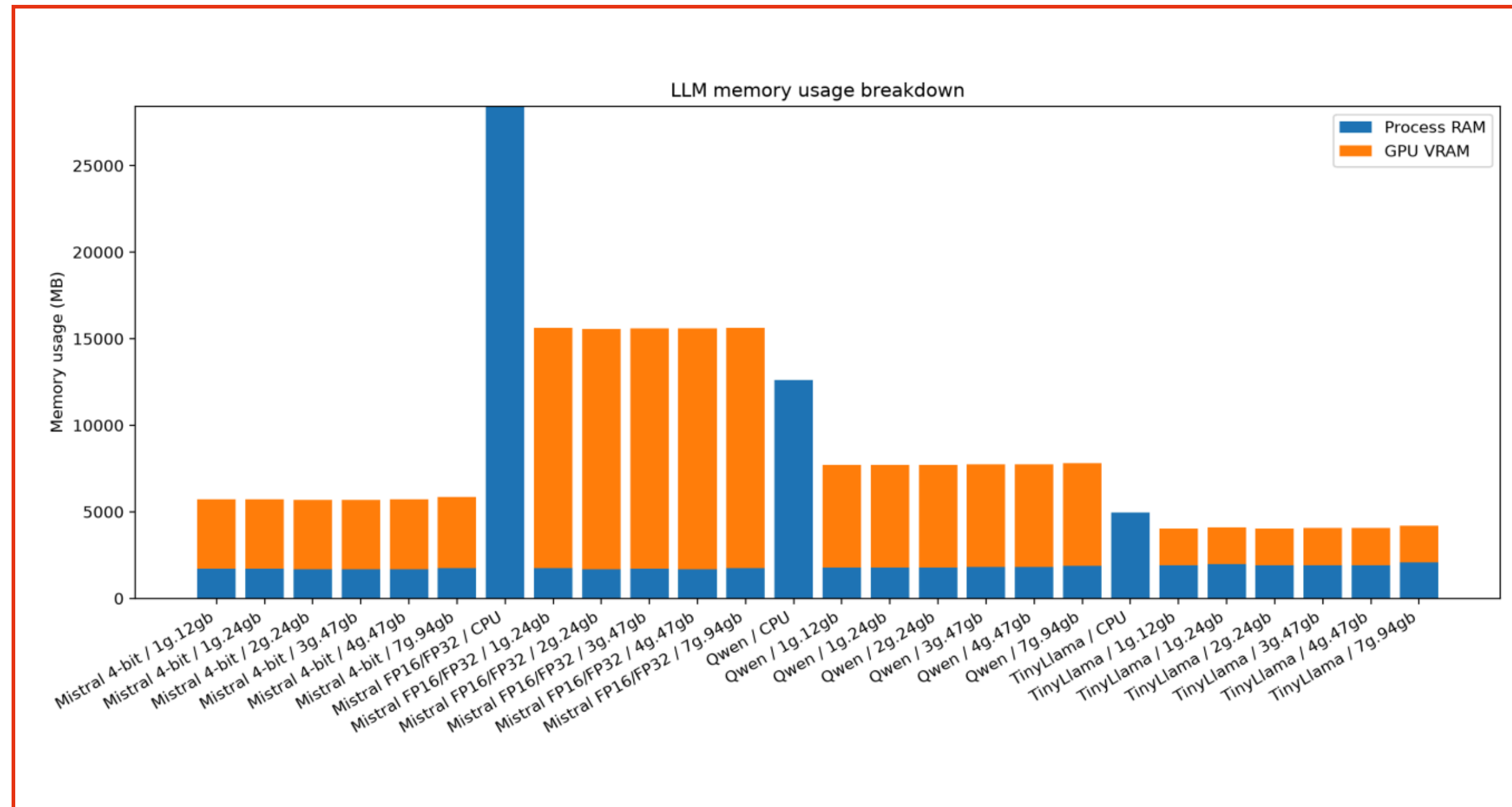
En inférence, les gains plafonnent assez vite

**CPU**

nettement plus lent

3g → 7ggain limité pour une
requête uniqueLa génération auto-régressive
limite le parallélisme disponible.

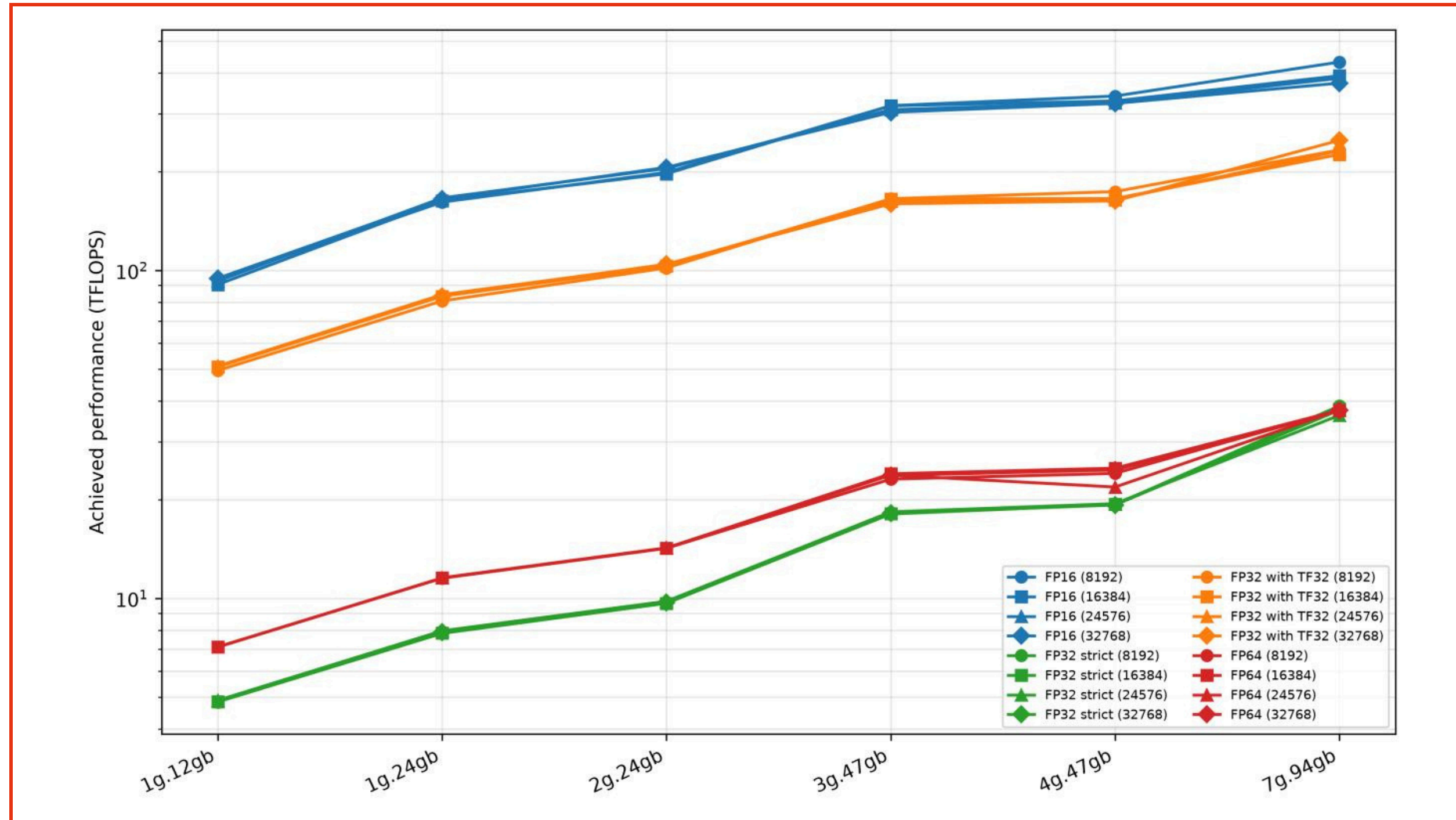
La mémoire du modèle détermine aussi le bon profil



- TinyLlama et Qwen fonctionnent sur les petits profils
- Mistral FP16 demande environ 15,6 Go de VRAM
- La quantification 4 bits ramène l'occupation autour de 5,7 Go
- Une petite instance peut donc servir davantage d'utilisateurs

Le bon profil dépend du débit et de la mémoire nécessaires.

Le calcul matriciel exploite mieux les ressources du GPU



FP16

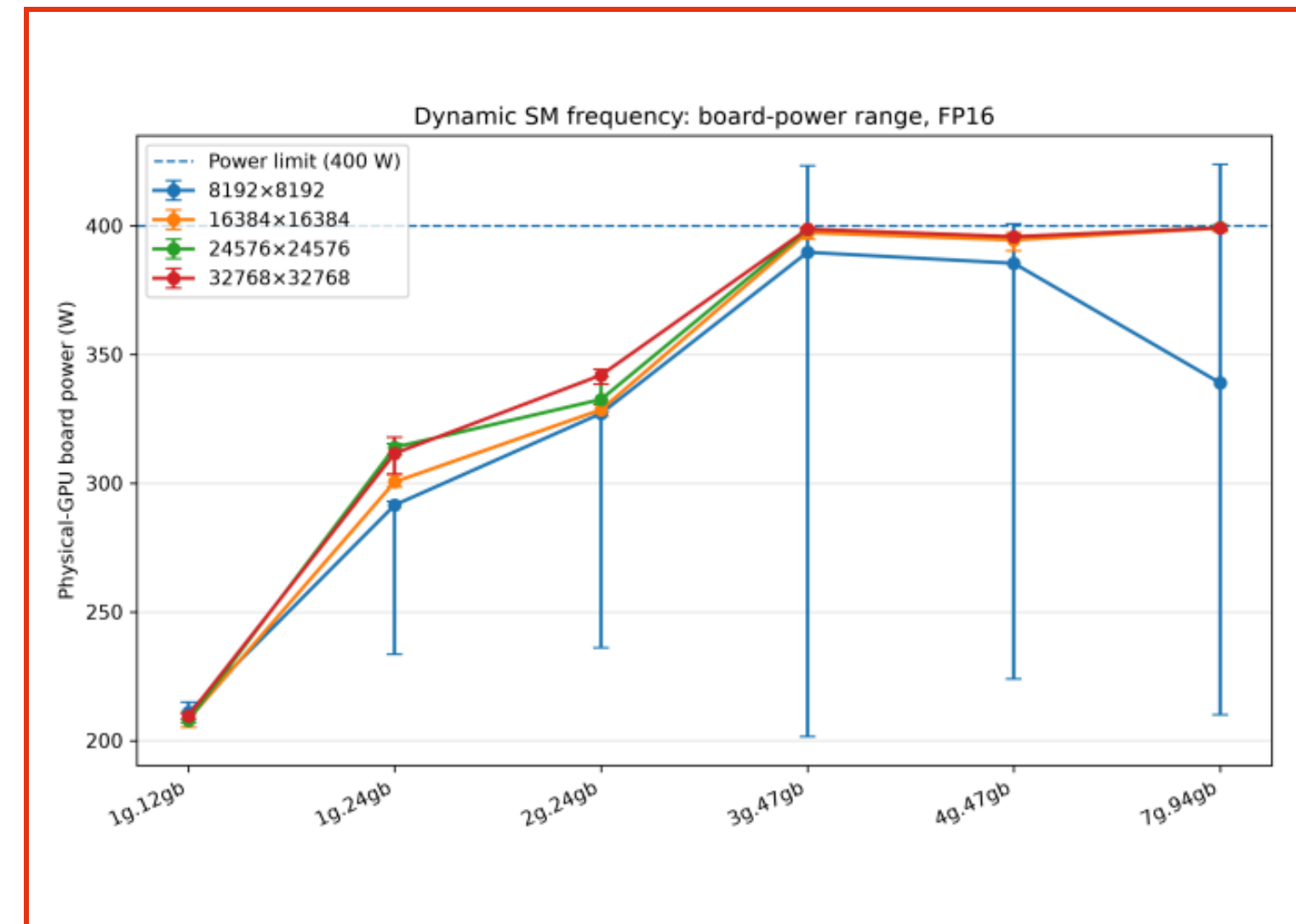
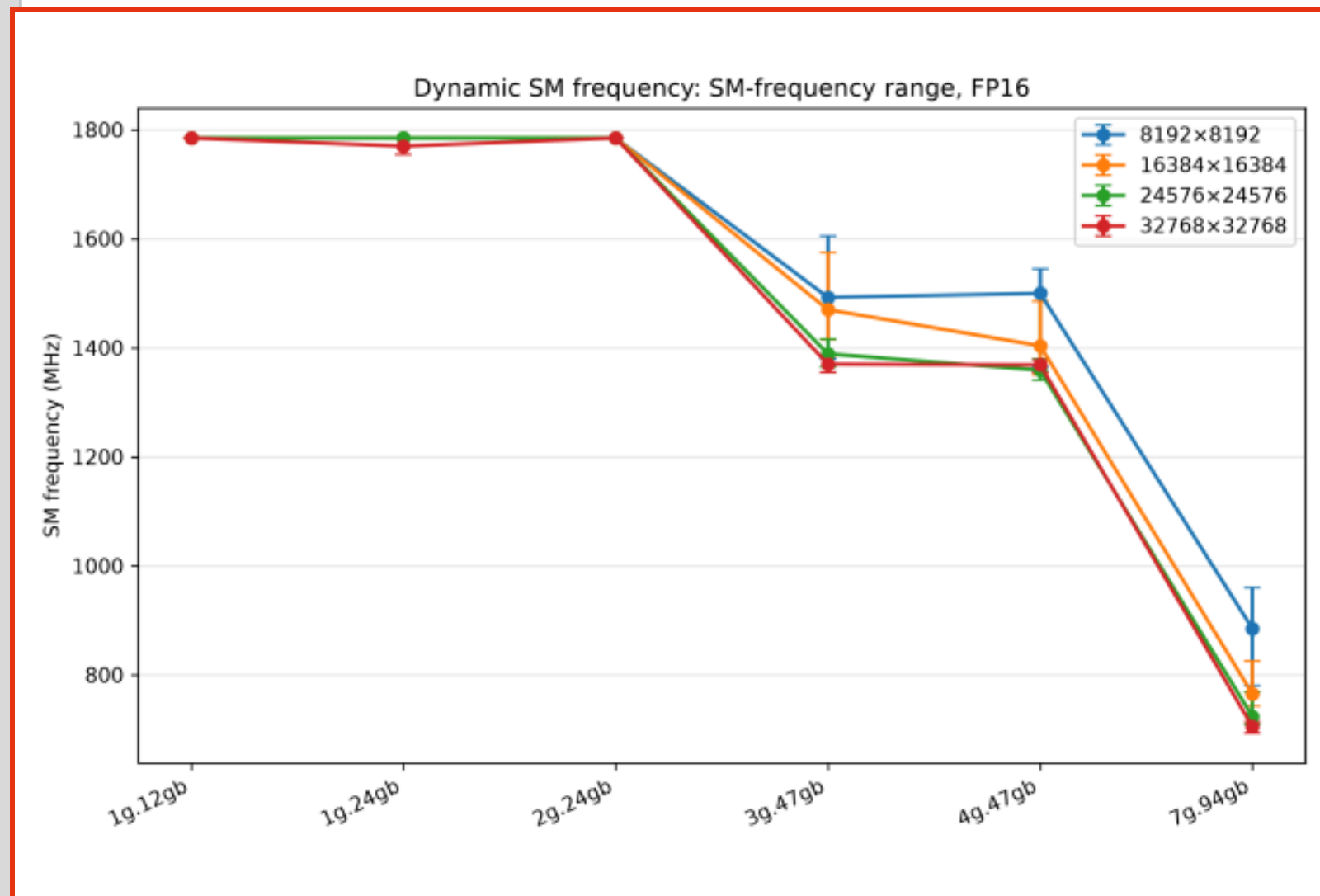
meilleure performance

TF32

accéléré par les Tensor
Cores

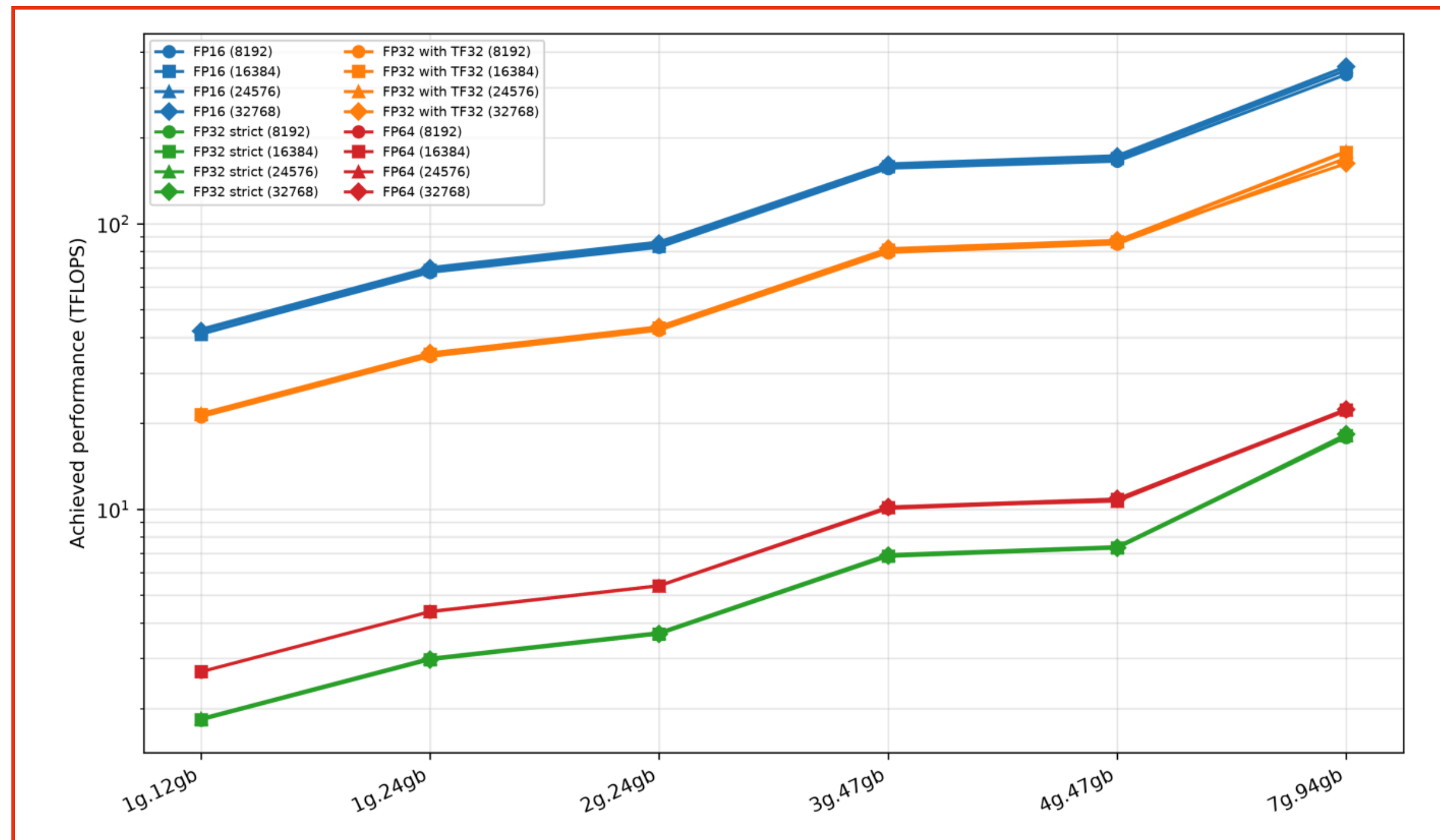
FP32 / FP64

Le profil maximal atteint presque la limite de puissance



Plus d'unités actives → puissance plus élevée → baisse automatique de fréquence

À 675 MHz, la comparaison devient plus équitable



- Même fréquence pour tous les profils
- Progression plus régulière avec le nombre de SM
- 7g.94gb proche du GPU complet pour cette charge

Le choix d'un profil MIG dépend d'abord de la charge

1

LLM

Une petite instance peut suffire pour une requête unique ou un modèle quantifié.

2

Calcul

Les grandes matrices bénéficient plus directement des unités supplémentaires.

3

Mesure

Fréquence et puissance doivent être relevées pour comparer équitablement.

Limite : NVIDIA mesure la puissance du GPU physique, pas celle de chaque instance MIG.

Conclusion : automatiser, isoler et mesurer

Automatiser

Une chaîne complète maintient le catalogue de ressources SLICES-RI.

Isoler

JupyterHub et Kubernetes fournissent des environnements reproductibles et sécurisés.

Mesurer

Les benchmarks MIG relient profil, charge, fréquence et performance.

Compétences développées

Python · Ansible · Docker · Kubernetes · GPU
Expérimentation reproductible · Recherche

Suite du parcours

Formation d'ingénieur à EURECOM

Inria

Merci de votre attention

Questions ?

Bohdan DYSHLEVYY · BUT3 Réseaux et Télécommunications